

처음 시작하는 파이썬 데이터분석 브리핑 문서

1. 데이터 분석의 개요

데이터 분석은 데이터를 이해하고, 현 상태를 파악하여 예측하며, 숨겨진 인과관계를 밝혀내는 체계적인 과정입니다. 이는 의사결정을 지원하고 문제를 해결하는 데 핵심적인 역할을 합니다.

1.1. 데이터 분석의 개념 및 목적

- **개념:** 데이터를 체계적으로 조사, 정리, 해석 및 시각화하여 유용한 정보를 얻는 과정입니다.
- **목적:** "데이터 분석은 데이터를 이해하기 위한 것입니다. 즉, 데이터가 나타내는 현 상태를 파악하여 예측하고, 숨겨진 이면의 데이터와 트리거가 된 데이터를 알아내 인과관계를 파악하는 것이 데이터 분석의 목적입니다."

1.2. 데이터 분석의 주요 특징 (과정)

1. **문제 정의:** 어떤 문제를 해결하려는지 명확히 정의하여 분석 방향을 설정합니다.
1. **데이터 수집:** 문제 해결에 필요한 데이터를 다양한 방식(내부 DB, API, 공공데이터, 설문조사, 웹 스크래핑/크롤링 등)으로 확보합니다.
 - **API (Application Programming Interface):** 소프트웨어 간 상호작용을 가능하게 하는 인터페이스로, 실시간 데이터 획득에 유용합니다.
 - **웹 스크래핑:** 웹 페이지에서 특정 데이터를 추출하는 작업입니다.
 - **웹 크롤링:** 인터넷상의 여러 웹 페이지를 자동으로 탐색하며 데이터를 수집하거나 웹 사이트 구조를 탐색합니다.
1. **데이터 전처리:** 수집된 데이터를 분석에 적합한 형태로 정제하고 변환합니다. (결측치, 이상치 처리 등)
1. **탐색적 데이터 분석 (EDA):** "데이터를 시각화하거나 통계적 기법 등을 사용해서 데이터를 이해하고 주요 패턴, 관계, 추세 등을 얻어냅니다."
1. **모델링 및 해석:** 예측, 분류 등을 위해 통계 모델이나 머신러닝 알고리즘을 활용하여 데이터의 규칙이나 경향성을 이해하고 의사결정에 활용합니다.
1. **결과 시각화 및 보고:** 분석 결과를 그래프, 차트, 대시보드 등으로 시각화하여 쉽게 전달합니다.

1.3. 데이터 분석의 주요 유형

- **기술적 분석 (Descriptive Analysis):** 과거 데이터 요약 및 설명으로 현 상태를 파악합니다. (예: 매출 상대 파악)
- **진단적 분석 (Diagnostic Analysis):** 특정 사건 발생 원인 및 관련 요인을 파악합니다. (예: 범죄 예방에 효과적인 가로등/보안등 유형 파악)
- **예측 분석 (Predictive Analysis):** 현재 데이터를 기반으로 미래를 예측합니다. (예: 노령화 지수 예측)
- **처방적 분석 (Prescriptive Analysis):** 최적의 의사결정을 위한 대안을 제시합니다. (예: 대중교통 배차 간격 및 노선 조정)

1.4. 데이터 분석을 통해 달성하는 것

- 의사결정 지원 및 문제 해결
- 효율성 및 생산성 향상
- 미래 예측
- 아이디어 창출
- 리스크 관리
- 고객 만족 개선
- 비용 절감
- 성과 측정/평가 및 혁신

1.5. 데이터 분석에 필요한 요소

1. **데이터 분석 목표 설정:** 분석의 성공 여부를 판단할 수 있는 명확한 결과물을 정의해야 합니다. 목표가 없으면 분석의 일관성을 잃을 수 있습니다.
1. **분석 결과 해석에 필요한 배경 지식 (실무 지식):** 통계학, 경제학, 정보기술, 수학 등의 학문적 지식과 분석 대상 데이터에 대한 실무 지식이 필수적입니다. 실무 지식 없는 분석은 "공허한 분석"이 될 수 있습니다.
1. **분석에 필요한 데이터:** 공공데이터, 웹/소셜 데이터 등 다양한 소스에서 데이터를 얻을 수 있습니다. (예: 공공데이터포털, Kaggle)
1. **데이터 분석 툴:** R, Python과 같은 무료 툴은 다양한 데이터(정형/비정형) 분석이 가능하며, 상호 보완적으로 사용하는 추세입니다.

2. 파이썬 개발 환경 설정

파이썬 기반 데이터 분석을 위해 주피터 노트북을 주로 사용하며, 이를 위한 개발 환경 설정 방법을 다

룹니다.

2.1. 파이썬 개발 환경 설정

- **주요 툴:** 주피터 노트북 (Jupyter Notebook)이 주로 사용됩니다.
 - **클라우드 기반:** 구글 코랩 (Google Colab) - 설치 없이 웹 브라우저에서 사용, 구글 계정(Gmail) 필요, Google Chrome 권장.
 - **설치형:** 아나콘다 주피터 노트북 (Anaconda Jupyter Notebook) - 아나콘다 설치 시 자동 설치되며, PC 운영체제에 영향을 받습니다.
- **주피터 노트북의 특징:** 데이터 분석 전용 툴로, 데이터 분석에 필요한 주요 기능과 라이브러리가 탑재되어 있습니다. 다목적 툴이 필요한 경우 PyCharm 등을 사용합니다.
- **구글 코랩 사용 순서:** 크롬 브라우저 및 구글 계정 준비 → 구글 코랩 사이트 접속 및 로그인 → 새 노트 생성 및 이름 변경 → 코드 실행 확인 → 라인 번호 추가 설정.
- **셀 관리:** 코드 작성 시 1개의 셀에 1개의 결과가 나오도록 작성하는 것이 좋으며, [코드] 셀과 [텍스트] 셀 추가/삭제가 가능합니다. [텍스트] 셀은 마크다운 문법을 지원합니다.
- **자동 저장 및 저장 위치:** 주피터 노트북 내용은 자동 저장되며, 'ipynb' 확장자를 갖습니다. 구글 코랩 주피터 노트북은 구글 드라이브의 '[Colab Notebooks]' 폴더에 저장됩니다.

2.2. 데이터 분석을 위한 프로젝트 구조 설정

- **프로젝트 구조의 중요성:** 코드 및 파일을 체계적으로 관리하기 위해 프로젝트 구조를 사용하는 것이 좋습니다.
- **기본 구조:**
 - **소스코드 파일 관리 폴더:** 프로그램의 동작을 수행하는 .ipynb, .py 파일 등. (구글 코랩에서는 [Colab Notebooks] 폴더)
 - **리소스 파일 관리 폴더:** 소스코드 파일을 제외한 모든 파일 (데이터 파일, 이미지 파일 등).
 - 구글 코랩의 기본 리소스 관리 폴더는 '/content' (일시적 메모리 공간)이며, 영구적인 디스크 공간은 구글 드라이브를 사용합니다. 구글 드라이브에 임의의 폴더를 생성하여 리소스 파일을 배치합니다. (예: 내 드라이브/pda_app)
- **권장 리소스 폴더 하위 구조:**
 - [data]: 정형/비정형 데이터 파일 (csv, xls, xlsx, txt, dat)
 - [img]: 입력에 사용되는 이미지 파일 (png, jpg)
 - [plots]: 정적 시각화 결과 파일 (png, jpg)
 - [html]: 인터랙티브 시각화 결과 파일 (html)
- **구글 코랩에서 프로젝트 구조 배치:**
 - 리소스 폴더를 구글 드라이브에 업로드합니다.
 - 소스코드 파일(.ipynb)은 GitHub에서 복제하여 구글 드라이브에 저장할 수 있습니다.
 - **구글 드라이브 사용 설정:** 구글 코랩은 Linux 기반이므로 구글 드라이브를 하드디스크처럼 사용하려면 '마운트 설정'이 필요하며, 이후 작업 디렉토리를 설정합니다.

3. 파이썬 기본 문법: 데이터 분석을 위한

데이터 분석의 순서와 각 단계에서 필요한 파이썬 기본 문법(변수, 타입, 제어문, 함수/모듈 등)을 학습합니다.

3.1. 데이터 분석 순서와 각 단계에서 기본 문법 사용

1. **목표 설정:** 분석할 목표(시각화된 분석 결과물)를 결정하는 단계로, 파이썬 기본 문법이 직접 사용되지 않습니다.
1. **데이터 준비 (로드):** 분석에 사용할 데이터를 확보하고 메모리로 로드하는 단계입니다.
 - **코드 구조:** 변수 = 값
 - **필요 문법:** 변수, 데이터 타입, 라이브러리(모듈) 로드 및 제공 함수, open() 문 등.
 - 정형 데이터(csv, xls, xlsx)는 pandas의 pd.read_csv(), pd.read_excel() 등으로 로드하고, 비정형 데이터(txt, dat)는 with open()과 for 문을 결합하여 로드합니다.
1. **데이터 처리:** 작업 대상 데이터를 처리하여 원하는 결과물(정제된 파일, 그래프 등)을 얻는 단계입니다.
 - **필요 문법:** 리스트, 튜플, 딕셔너리 다루기, 제어문, 라이브러리 제공 함수, 데이터프레임 함수, 시각화 함수 등.

3.2. 데이터 분석에 필요한 기본 문법

1. 값 저장소: 변수

- **개념:** 프로그램 수행 중 발생하는 값을 임시로 저장하는 메모리 공간입니다.
- **선언:** 변수명 = 값 형태로 값을 할당하여 선언합니다. 할당되는 값의 타입에 따라 변수 타입이 결정됩니다.
- **제거:** del 변수명을 사용하여 메모리에서 제거하여 리소스를 반환하고 성능을 향상시킵니다.

- **식별자 표기 규칙:** 클래스명, 함수명, 변수명 등에 사용되는 이름입니다. 첫 글자는 문자로 시작하며 숫자, 언더바(_)를 조합하여 사용합니다.
 - **카멜 표기법 (CamelCase):** 단어 연결 시 다음 단어의 첫 글자를 대문자로 표기 (예: `userAge`). 주로 클래스명에 권장됩니다.
 - **스네이크 표기법 (snake_case):** 단어 연결 시 언더바(_) 사용 (예: `user_age`). 파이썬에서 선호하는 표기법입니다.

1. 저장되는 값의 타입

- **1개 값 타입 (기본 데이터 타입):**
 - `int` (정수형): 10, -5
 - `float` (실수형): 3.14, -0.5
 - `complex` (복소수형): 3 + 4j
 - `str` (문자열): 'Hello', "Python"
 - `bool` (불린): True, False
- **여러 개 값 저장 타입 (컬렉션 데이터 타입):** 데이터 분석에서 자주 사용됩니다.
 - `list` (리스트): [1, 2, 3]. 순차적으로 나열하며 대괄호 []로 표현. 원소 값 변경 가능.
 - 리스트 컴프리헨션: [수식 for 변수 in 값_목록 if 조건] (간편하고 빠름).
 - `append()` 메서드: 리스트에 원소 추가.
 - 인덱싱/슬라이싱: `리스트_변수명[인덱스]`, `리스트_변수명[시작:끝]`.
 - `range()` 함수: 등차수열(값 목록) 생성. `range(시작값, 끝값, 증가값)`.
 - `enumerate()` 함수: 인덱스와 값을 함께 반복 추출.
 - `tuple` (튜플): (1, 2, 3). 소괄호 ()로 표현. 원소 값 변경 불가능 (보안에 유리).
 - `dict` (딕셔너리): {'id': 'aaaa', 'pass': 80}. 키와 값의 쌍으로 저장하며 중괄호 {}로 표현. 키를 사용하여 값을 얻거나 변경, 추가 가능.
 - `items()` 메서드: 키와 값을 함께 반복 추출.
 - `keys()` 메서드: 딕셔너리의 키 추출.

1. 값 타입 변환 및 확인 함수

- **타입 변환 (캐스팅) 함수:**
 - `int()`, `float()`, `str()`: 기본 데이터 타입 변환. 숫자 + 문자열 연산 시 `str()` 필수.
 - `list()`, `dict()`, `tuple()`: 컬렉션 데이터 타입 변환.
 - `zip()` 함수: 개별 값/값 목록을 순서대로 튜플 쌍으로 묶어줍니다.
- **타입 확인 함수:** `type(변수)` (일반 변수), `데이터프레임.dtypes` (데이터프레임 열 변수).

1. 제어문

- **if 문 (조건문):** 주어진 조건식의 결과(True/False)에 따라 분기. `if-elif-else` 구조, 인라인 `if` 문 (참 `if` 조건 `else` 거짓).
 - 조건식은 관계 연산자(==, !=, <, >, <=, >=)와 함께 사용되며, 범주형/수량형 변수에 따라 다르게 구성됩니다.
- **for 문 (반복문):** 값 목록의 데이터 수만큼 또는 주어진 횟수만큼 반복 작업 처리. `for` 제어변수 `in` 값_목록_변수/`range()`:
- **with 문:** 파일을 열고 닫는 작업을 자동으로 수행하여 리소스 관리를 효율적으로 합니다. `with open("파일명", "모드") as 파일_객체:`

1. 함수/모듈

- **함수:** 특정 작업을 수행하도록 미리 작성된 코드.
 - **내장 함수:** 시스템이 제공하는 함수 (예: `input()`, `print()`).
 - **사용자 정의 함수:** 사용자가 직접 만드는 함수 (`def` 함수명(매개변수):).
 - **람다 (lambda) 식:** 이름 없는 함수로, 간결하게 함수를 정의할 때 사용됩니다. (`lambda` 매개변수: 함수_내용).
- **모듈 (라이브러리, 패키지):** 값 및 데이터 처리에 필요한 함수를 제공합니다.
 - **로드:** `import 라이브러리명` 또는 `import 라이브러리명 as 별명`.
 - **설치:** `pip install 라이브러리` (주피터 노트북에서는 `!pip install 라이브러리`).

4. 파이썬을 사용한 데이터 분석 - 주요 라이브러리, 데이터 로드, 전처리, 시각화, 통계 분석

4.1. 데이터 분석에 필요한 주요 라이브러리 데이터 분석에서는 데이터프레임을 빠르게 처리하는 고속 연산 및 과학 계산 라이브러리, 그리고 시각화 라이브러리가 필수적입니다.

1. NumPy (넘파이):

- **개념:** 파이썬에서 과학 계산을 위한 핵심 라이브러리. 수치 데이터를 고속으로 처리하는 다차원 배열 객체(`ndarray`)를 제공합니다.

- **주요 특징:** ndarray, 벡터 연산 지원, 다양한 수학 함수 제공, C/C++ 및 Fortran 통합 가능.
- **장점:** 빠른 연산 속도, 메모리 효율성, 수학/과학/머신러닝에 필수적입니다.
- **사용법:** import numpy as np

1. SciPy (사이파이):

- **개념:** 과학 계산 및 공학적 연산을 위한 파이썬 라이브러리로, NumPy를 확장한 형태입니다.
- **주요 기능:** 선형대수 (scipy.linalg), 최적화 (scipy.optimize), 통계 (scipy.stats), 신호 처리 (scipy.signal), 보간법 (scipy.interpolate) 등.
- **사용법:** import scipy as sp (주로 numpy와 함께 사용)

1. Matplotlib (맷플롯립):

- **개념:** 파이썬에서 데이터를 시각화하는 기본적인 라이브러리. 주로 matplotlib.pyplot 모듈을 사용합니다.
- **주요 특징:** 다양한 그래프 (선, 산점도, 히스토그램, 막대 그래프 등), 세밀한 조정 가능, 다양한 데이터 형식 지원, 3D 플롯 및 애니메이션 지원.
- **사용법:** import matplotlib.pyplot as plt (약어 plt 사용). 그래프를 파일로 저장하거나 화면에 표시할 수 있습니다.
- **한글 깨짐 방지:** 그래프에서 한글이 깨지는 것을 방지하기 위해 폰트 설정을 전역으로 지정해야 합니다. (OS별 설정 방법 다름)

1. Seaborn (시본):

- **개념:** Matplotlib을 기반으로 한 고급 데이터 시각화 라이브러리로, 통계 기반 시각화에 강점을 가집니다.
- **주요 특징:** Pandas와 연동, 통계 기반 시각화 (히스토그램, KDE, 회귀선), 적은 코드로 세련된 시각화 가능.
- **사용법:** import seaborn as sns (주로 numpy, scipy, pandas, matplotlib와 함께 사용).

4.2. 실무 데이터 로드 및 데이터프레임 다루기

데이터프레임은 행(데이터)과 열(변수)로 이루어진 2차원 데이터 구조로, 데이터 분석의 핵심입니다.

1. 데이터 파일 로드 개요:

- **정형 데이터:** CSV, Excel 파일은 pandas의 pd.read_csv(), pd.read_excel() 함수를 사용합니다.
 - pd.read_csv("파일명.csv", encoding="cp949") (Windows ANSI 인코딩의 경우)
 - pd.read_excel("파일명.xlsx", sheet_name=0) (시트 번호 또는 시트명 지정)
- **토이 데이터:** sklearn.datasets에서 제공하는 소규모 데이터셋을 로드할 수 있습니다. (예: load_iris())
- **반정형 데이터:** API 응답으로 제공되는 XML/JSON 형식의 데이터는 json.load() (JSON) 또는 xml.etree.ElementTree의 ET.parse() (XML)를 사용하여 로드합니다.
- **비정형 데이터:** TXT, DAT 파일은 with open("파일명", "r") as f: 문과 for 문을 결합하여 로드한 후 데이터프레임으로 변환합니다.
- **파일 저장:** df.to_csv("파일명.csv", index=False, encoding="cp949")와 같이 to_타입() 함수를 사용합니다.

1. 데이터프레임 다루기:

- **데이터프레임 생성:** pd.DataFrame()으로 딕셔너리를 변환하거나, 파일 로드 함수를 통해 생성합니다.
- **데이터프레임/시리즈 타입 확인 및 변환:**
 - type(변수): 일반 변수 타입 확인.
 - df.dtypes: 데이터프레임의 모든 열 변수 타입 확인.
 - df['변수'].astype(str): 특정 변수 타입 변환.
 - pd.to_datetime(df['변수']): 날짜/시간 타입으로 변환.
- **데이터프레임 내용 및 구조 파악:**
 - df.head() / df.tail(): 데이터 앞/뒤 일부 확인.
 - df.info(): 데이터프레임의 구조 (변수 타입, 결측치 등) 확인.
 - df.shape: 데이터프레임의 행과 열 개수 확인.
 - df.describe(): 수량형 변수의 요약 통계량 (평균, 표준편차, 사분위수 등) 확인.
 - df['변수'].value_counts(): 범주형 변수의 빈도표 확인.
 - df['변수'].isnull().sum(): 특정 변수의 결측치 개수 확인.
 - df.columns / df.values: 변수명 / 데이터 값 얻기.
- **데이터프레임의 기본적인 연산 방법:** 변수 단위로 연산이 적용되며, NaN 값은 연산 전

에 처리해야 합니다.

- **주요 통계 함수:** sum(), mean(), count(), mode(), min(), max(), median(), quantile(), var(), std(), round() 등.
- **함수 적용:** df['변수'].apply(함수) 또는 df[['변수1', '변수2']].apply(함수)를 사용하여 변수에 함수(람다식 포함)를 적용합니다.

4.3. 데이터 전처리: 분석 데이터 준비 데이터 전처리는 데이터 분석에 필요한 데이터를 필터링, 변수 선택, 생성, 결측치/이상치 처리 및 스케일링/인코딩 등의 작업을 통해 분석 데이터를 준비하는 과정입니다.

1. 변수 (필드) 선택:

- df["변수명"] / df[["변수1", "변수2"]]: 변수명으로 선택.
- df.loc[:, "변수명"] / df.loc[:, ["변수1", "변수2"]] / df.loc[:, "변수1":"변수2"]: loc를 사용하여 변수명으로 선택 (행/열 라벨 기반).
- df.iloc[:, 인덱스_번호] / df.iloc[:, [인덱스1, 인덱스2]] / df.iloc[:, 인덱스1:인덱스2]: iloc를 사용하여 인덱스 번호로 선택 (정수 위치 기반).
- df[df.columns[시작:끝]]: 컬럼 인덱스를 사용하여 변수 선택.

1. 값 1개 추출: df.at[행_인덱스, 열_인덱스] (loc보다 빠름).

1. 인덱스 지정과 복귀: df.set_index("인덱스로_지정할_필드명") (특정 변수를 행 인덱스로), df.reset_index() (기본 인덱스로 복귀).

1. 새 필드 추가 (파생변수 생성): df["새필드명"] = 수식 (함수, 연산식, 람다식 등).

1. 필터링 (데이터 추출): df[조건] (조건을 만족하는 행만 추출). 여러 조건은 &(and), |(or) 연산자로 연결.

1. 조인 (Join): df1.join(df2.set_index("인덱스_필드"), on="인덱스_필드") (변수(열) 결합).

1. 변수명 변경: df.rename(columns={"원변수명": "새변수명"}, inplace=True).

1. 바인딩 (Binding): pd.concat([df1, df2], ignore_index=True) (데이터(행) 결합), pd.concat([df1, df2], axis=1, join="inner") (변수(열) 결합).

1. 그룹화 (Grouping): df.groupby("그룹화_변수명").통계함수() 또는 .agg(["통계함수"]) (집단별 요약 통계량).

1. 정렬 (Sorting): df.sort_values(by="정렬기준_변수", ascending=True/False) (오름차순/내림차순 정렬), pd.Categorical()을 사용하여 사용자 정의 정렬도 가능합니다.

1. 행, 열 (변수) 제거: df.drop(df.index[행번호], inplace=True) (행 제거), df.drop(labels=["변수명"], axis=1, inplace=True) (열 제거).

1. 데이터프레임 구조 변환:

- df.transpose(): 행과 열 바꿈.
- pd.melt(df, id_vars=["기준_변수"]): 여러 변수를 1개의 변수로 (집단 비교).
- df.pivot(index="", columns="", values=""): 1개의 변수를 여러 개의 변수로 (관계 파악).

4.4. 결측치/이상치 처리, 스케일링/인코딩 방법

1. 결측치 (Missing Values) 처리:

- 확인: df.isnull().sum(), df['변수'].notnull().
- 처리:
 - df.dropna(): 결측치가 있는 행 제거.
 - df['변수'].fillna(대체값): 결측치를 특정 값 (0, 평균, 중앙값, 이전 값 ffill, 이후 값 bfill 등)으로 대체.

1. 이상치 (Outliers) 처리:

- 확인: IQR (Interquartile Range) 기준 ($Q1 - 1.5/IQR \sim Q3 + 1.5/IQR$)을 사용하여 통계적으로 정상 범위를 벗어난 값들을 식별합니다.
- 처리: 이상치를 결측치로 대체 후 제거하거나 다른 값(평균 등)으로 보정합니다.

1. 수치형 변수 정규화 스케일링 (Normalization Scaling):

- 목적: "데이터의 범위(크기)를 조정하는 전처리 과정으로, 머신러닝과 통계 분석에서 모델 성능 향상과 학습 안정성 확보를 위해서 사용합니다." (변수 간 단위 차이 보정, 경사 하강법 안정성 향상, 거리 기반 알고리즘 정확도 향상 등)
- 방법:
 - MinMaxScaler(): 0~1 사이로 변환 (이상치에 민감).
 - StandardScaler(): 평균 0, 표준편차 1로 표준화 (이상치 영향 적음).

1. 범주형 변수 값을 숫자로 변환하는 레이블/원-핫 인코딩 (Encoding):

- 목적: "범주형 변수의 값을 수량화하는 방법으로 머신러닝이나 통계 분석에서 모델이 수치형 입력을 필요로 할 때 필수적인 전처리 과정입니다."
- 방법:

- `LabelEncoder()` (레이블 인코딩): 각 범주에 정수 값 할당 (예: A→0, B→1, C→2). 순서가 있는 범주에 적합.
- `pd.get_dummies()` (원-핫 인코딩): 각 범주를 이진 벡터로 변환 (예: A→[1,0,0]). 범주 간 순서가 없을 때 적합.

4.5. 데이터 시각화: 탐색적 데이터 분석 시각화는 데이터의 주요 분포와 모양을 파악하여 현 상황을 이해하는 탐색적 데이터 분석의 핵심입니다.

1. **Pandas 라이브러리를 사용한 데이터프레임 시각화:** `df.plot()`과 같은 데이터프레임 메서드를 사용하여 Matplotlib 기능을 활용합니다.

- **시각화 방법 개요:** 변수의 개수와 종류에 따라 적합한 시각화 기법을 선택합니다.
 - 수량형 1개: 히스토그램 (도수별 값 분포).
 - 범주형 1개: 빈도 막대 그래프, 원형 그래프 (범주별 값 분포).
 - 수량형 2개: 산점도, 히트맵, 시계열 그래프 (두 변수 간 관계).
 - 범주형 1개, 수량형 1개: 막대 그래프, 병렬 상자그림 (집단 비교).
 - 산점도 행렬: 여러 수량형 변수의 모든 산점도 조합.
- **그래프 종류별 시각화 방법:**
 - 원형 그래프 (`df.plot.pie()`): 전체 중 각 집단이 차지하는 비율.
 - 막대 그래프 (`df.plot.bar()`): 범주형 변수의 각 집단 값 비교 (빈도, 합계 등).
 - 히스토그램 (`df.plot.hist()`): 수량형 변수의 도수 빈도. 서브플롯으로 여러 변수 동시 시각화 가능.
 - 산점도/산포도 (`df.plot.scatter()`): 두 수량형 변수의 관계.
 - 산점도에 선형 회귀선 추가 (`sns.regplot()`): 값의 방향 예측.
 - 상관관계 (`df.corr()`): 두 수량형 변수의 관계를 값으로 표현 (피어슨, 스피어만, 칸달).
 - 상관관계 행렬 히트맵 (`sns.heatmap()`): 여러 수량형 변수의 상관관계를 시각화.
 - 산점도 행렬 (`scatter_matrix()` 또는 `sns.pairplot()`): 여러 수량형 변수의 모든 산점도 조합을 시각화.
 - 히트맵 (Hexbin) (`df.plot.hexbin()`): 데이터 수가 많을 때 분포 파악.
 - 선 그래프 (`df.plot.line()`): 시계열 데이터의 추이. 선형회귀선 추가 가능.
 - 상자그림 그래프 (`df.boxplot()` 또는 `sns.boxplot()`): 범주형-수량형 변수의 집단 간 값 비교. `sns.stripplot()` 또는 `sns.swarmplot()`으로 산점도 추가 가능.
 - 워드클라우드 (`WordCloud()`): 텍스트에서 언급되는 단어의 빈도를 시각적으로 표현.

1. **Folium 라이브러리를 사용한 공간 데이터 시각화:** 지리 정보 데이터를 지도 위에 시각화합니다.

- **개념:** Python의 데이터 랭글링과 Leaflet.js 라이브러리의 매핑 강점을 결합하여 상호작용적인 지도를 생성합니다. GeoJSON 데이터 지원.
- **사용법:** `import folium`.
- **지도 표시:** `folium.Map([위도, 경도], zoom_start=확대_정도)`.
- **마커 및 툴팁/팝업 표시:** `folium.Marker([위도, 경도], popup, tooltip, icon).add_to(map)`.
 - 팝업 상자 너비 조정: `folium.Popup(내용, min_width, max_width)`.
 - 마커 아이콘 변경: `folium.Icon("색상", icon="종류")` 또는 `folium.CustomIcon("이미지", icon_size)`.
 - 웹사이트 연동, 반경 원 표시: `folium.Circle()`.
- **GeoJSON 데이터를 사용한 행정구획 표시:**
 - GeoJSON 파일 다운로드 및 로드: `urllib.request.urlretrieve()`, `json.load()`.
 - 지도에 행정구획 표시: `folium.GeoJson(GeoJSON_데이터, name='레이어_이름').add_to(map)`.
 - 특정 지역만 추출하여 표시 가능합니다.
 - 지도 맵 타일 변경: `folium.Map(tiles="CartoDB positron")`.
- **지도에 공간 데이터 추가 (마커 클러스터):** `MarkerCluster().add_to(map)`를 사용하여 비슷한 위치에 많은 마커를 그룹화하여 표시.

4.6. 통계적 데이터 분석 통계적 데이터 분석은 통계 기법을 활용하여 데이터를 해석하고 의미 있는 정보를 얻어내기 위한 과정입니다.

1. **통계적 데이터 분석 개요:**

- **개념:** "현 상황을 이해하거나 미래를 예측하기 위해 통계 기법을 활용하는 분석 과정입니다."
- **목적:** 현 상황 파악, 미래 예측, 인과관계 파악.

- **분석 단계:** 문제 정의 → 데이터 수집 → 데이터 전처리 → 탐색적 데이터 분석 (EDA) → 통계 분석 기법 적용 → 해석 및 의사결정.
- **변수 종류에 따른 분석 기법:**
 - 1개 수량형 변수: 히스토그램, 요약 통계량, t-검정 (독립, 대응, 단일 표본).
 - 1개 범주형 변수: 빈도표 (crosstab()), 막대/원형 그래프, 이항 검정.
 - 2개 수량형 변수: 산점도, 상관계수, 단순 회귀분석, 라쏘/릿지 회귀분석.
 - 1개 범주형, 1개 수량형 변수: 병렬 상자그림, 분산분석 (ANOVA).

1. 온실가스 데이터 분석: 회귀분석, 주기성을 갖는 시계열 데이터 예측

- **회귀분석:** 독립 변수(x)가 종속 변수(y)에 미치는 영향을 분석하여 값을 예측하거나 인과관계를 파악하는 통계 기법입니다.
- **데이터 로드 및 탐색/전처리:** 1999-2023년 온실가스 배경농도 데이터 (1999-2023_ghgs.csv)를 로드하여 구조 및 요약 통계량 확인, 결측치 제거(dropna()), 수량형 변수 데이터프레임 생성.
- **온실가스 배경농도 값 현 상태와 예측:**
 - **CO2, CH4 상관계수:** df.corr()를 사용하여 두 온실가스 간의 관계를 수치로 확인.
 - **선형 회귀선 포함 산점도:** sns.regplot()으로 시각화.
 - **선형 회귀식:** sklearn.linear_model.LinearRegression을 사용하여 기울기 (model.coef_)와 절편(model.intercept_)을 구합니다.
 - **선형 회귀분석 (최소제곱법 ols() / sm.OLS()):** statsmodels.formula.api.ols() 또는 statsmodels.api.OLS()를 사용하여 회귀모형을 학습하고 summary()로 상세 결과를 확인합니다.
 - **R-squared:** 설명력. 높을수록 예측 모형 적합.
 - **F-statistic / Prob(F-statistic):** 회귀식 전체의 유의성 검정. p-value가 낮으면 유의미.
 - **회귀계수 (coef):** 독립변수가 종속변수에 미치는 영향.
 - **Durbin-Watson:** 잔차의 자기상관 확인. 2에 가까우면 독립.
 - **Cond. No. (조건수):** 다중공선성 위험 지표. 10,000 이상이면 잠재적 위험.
 - **값 예측:** result.predict(X_new)를 사용하여 새로운 독립변수 값에 대한 종속변수 값을 예측합니다.
 - **정규화 스케일링:** StandardScaler()로 데이터를 표준화하여 다중 회귀분석 및 머신러닝에 활용합니다.
- **온실가스들 간의 관계 분석:**
 - **상관계수 행렬:** df.corr()로 모든 수량형 변수 간의 상관계수를 계산.
 - **상관계수 행렬 히트맵:** sns.heatmap()으로 시각화하여 상관관계 패턴을 시각적으로 확인.
 - **산점도 행렬:** sns.pairplot()으로 모든 변수 쌍의 산점도와 회귀선을 시각화.
 - **다중 회귀분석 (최소제곱법 sm.OLS()):** 여러 독립변수가 하나의 종속변수에 미치는 영향을 분석. 다중공선성 문제가 발생할 수 있으며, 이를 해결하기 위해 VIF (분산팽창계수) 확인, 라쏘/릿지 회귀 등을 사용합니다.
 - **다중공선성:** 독립변수들 간에 높은 상관관계가 있을 때 발생하는 문제. 회귀계수의 불안정성, 해석 어려움, 예측력 감소 등의 문제가 있습니다.
 - **다중공선성 탐지:** 상관계수 행렬, VIF, 조건수.
 - **다중공선성 해결:** 변수 제거, PCA, 정규화 회귀 (라쏘, 릿지), 변수 변환.
 - **라쏘 회귀 (Lasso()):** make_pipeline(StandardScaler(), Lasso(alpha=0.1))을 사용하여 다중공선성을 줄이고 중요한 변수를 식별. alpha 값으로 규제강도 조절.
 - **릿지 회귀 (Ridge()):** make_pipeline(StandardScaler(), Ridge(alpha=1.0))을 사용하여 다중공선성을 줄이고 과적합 방지.
- **주기성을 갖는 시계열 온실가스 CO2 배경농도 추이 파악과 예측:**
 - **ARIMA (Autoregressive Integrated Moving Average) 모델:** 비계절적 시계열 데이터의 추이 및 예측.
 - **SARIMA (Seasonal ARIMA) 모델:** "주기성(Seasonality)이라는 데이터가 시간이 지남에 따라 일정한 주기로 반복되는 패턴으로 월, 연도, 계절 등과 관련된 반복 현상이 있을 때는 SARIMA 모델을 사용합니다." (온실가스 CO2 배경농도처럼 주기성을 갖는 데이터에 적합)
 - **SARIMAX 모델:** statsmodels.tsa.statespace.sarimax.SARIMAX()를 사용하여

SARIMA 모델을 구현합니다. order=(p,d,q)와 seasonal_order=(P,D,Q,s) 파라미터를 설정합니다.

- **모델 학습:** model.fit().
- **예측:** results.get_forecast(steps=12) (향후 12개월 예측), forecast.predicted_mean (예측값), forecast.conf_int() (신뢰구간).
- **성능 평가:** results.summary()를 통해 AIC, BIC, 계수 유의성, 잔차 진단(자기상관, 정규성, 이분산성) 등을 확인합니다. SARIMA 모델이 ARIMA 모델보다 더 나은 예측 성능을 보입니다.

1. 서울시 하수처리장 데이터 분석: 분산분석 (ANOVA)

- **분산분석 (ANOVA, Analysis of Variance):** "두 개 이상의 그룹 간의 평균 차이를 비교하는 데 사용되는 통계 기법으로 집단을 비교할 때 사용합니다."
- **목표:** 서울시 하수처리장별 c19바이러스 농도 데이터의 현 상황 파악, 하수처리장 간의 관계 분석, 하수처리장 집단 비교 분석.
- **데이터 로드 및 탐색/전처리:** 2024년 서울시 하수처리장 코로나바이러스 농도 데이터 (2024년서울시_하수처리장_코로나바이러스농도.xlsx)를 로드하여 구조 및 요약 통계량 확인. (이상치 보정은 질병 확산의 중요성을 고려하여 진행하지 않음).
- **하수처리장 바이러스 농도 값 현 상태와 예측:**
 - **시계열 그래프:** df.plot.line()으로 시점에 따른 모든 하수처리장의 바이러스 농도 추이 파악.
 - **상관계수 및 선형 회귀분석:** 특정 두 하수처리장(예: 서남, 난지) 간의 상관관계 및 선형 회귀 분석을 통해 관계를 파악하고 예측 모델을 구축합니다. (위 온실가스 데이터 분석과 동일한 방법론 적용)